

Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий

Кафедра вычислительной техники

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №3

Преподаватель

Студент

КИ22-07Б, 032215583
номер группы, зачетной книжки

подпись, дата

подпись, дата

Д. С. Романова

инициалы, фамилия

А. М. Коробков

инициалы, фамилия

СОДЕРЖАНИЕ

Введение.....	3
Ход работы.....	4
Заключение	8
Приложение А	9

ВВЕДЕНИЕ

Описание полученного задания:

1. размещение исходного кода, полученного при выполнении работы 1 в git-репозитории;
2. анализ решения, полученного при выполнении работы 1 с использованием инструментальных средств:
 - линтера;
 - статического или динамического анализатора или профайлера.
1. улучшение существующего кода (рефакторинг), при этом каждое изменение фиксируется в виде коммита в репозитории.

Ход работы

Для выполнения практической работы №3 необходимо разместить исходный код программы в git-репозиторий. Для этого создадим репозиторий, используя сервис контроля версий программ GitHub. Ссылка на репозиторий, содержащий исходный код программы приведена в приложении А. Далее, по ходу выполнения практической работы, история изменения исходного кода программы будет отображаться внутри коммитов в репозитории.

Для анализа решения используем статический анализатор кода Clang-Tidy. Данный инструмент представляет собой не только статический анализатор кода, но и линтер. Для его применения опишем workflow внутри репозитория. Для этого:

1. создадим папку `/.GitHub/workflows`;
2. внутри этой папки создадим файл `linter` с расширением `.yaml` и опишем workflow;

Для описания workflow необходимо задать имя, событие на срабатывание, а также само действие. Пример workflow, использующее линтер Clang-tidy, представлен на рисунке 1.

```

1  name: Code Linter
2
3  on:
4    push:
5      branches: ["dev", "main"]
6    pull_request:
7      branches: ["main", "dev"]
8
9  jobs:
10   clang-tidy:
11     runs-on: windows-latest
12
13     steps:
14       - name: Checkout repository
15         uses: actions/checkout@v4
16
17       - name: Setup Visual Studio
18         uses: ilammy/msvc-dev-cmd@v1
19
20       - name: Install Clang-Tidy
21         shell: pwsh
22         run: |
23           choco install llvm -y
24           refreshenv
25
26       - name: Run Clang-Tidy
27         shell: pwsh
28         run: |
29           $ClangTidy = "C:\Program Files\LLVM\bin\clang-tidy.exe"
30           if (Test-Path $ClangTidy) {
31             # Получаем пути к include MSVC и Windows SDK
32             $IncludePaths = "$env:VC_IncludePath;$env:WindowsSDK_IncludePath"
33
34             Get-ChildItem -Path "opppo_task1" -Recurse -Include "*.cpp","*.h" | Where-Object { $_.FullName -notmatch "opppo_task1.tests" } | ForEach-Object {
35               Write-Host "Analyzing: $($_.FullName)"
36               & $ClangTidy $_.FullName -- -x c++ -std=c++17 -Iopppo_task1.core -I$IncludePaths
37             }
38           } else {
39             Write-Host "Clang-Tidy not found, skipping linting"
40           }
41           continue-on-error: true

```

Рисунок 1 – Пример описания workflow

После этого создадим еще несколько workflow, которые будут применяться на push в dev ветку репозитория. В данном случае были описаны workflow для сборки проектов и выполнения тестов, а также применение форматирования текста исходного кода при помощи утилиты Clang-Format.

После описания нужных workflow нужно отследить лог линтера, который подскажет, где в коде есть синтаксические и логические ошибки. После прочтения лога линтера было принято решение организовать рефакторинг кода. Для этого проект был разбит на несколько подпроектов, которые включали в себя следующие проекты:

1. Проект `opppo_task1` – нужен для сборки исполняемого файла;
2. Проект `opppo_task1.core` – нужен для сборки статической библиотеки классов и методов;
3. Проект `opppo_task1.tests` – нужен для сборки динамической библиотеки Unit-тестов.

Структура проектов, описанных выше, представлена на рисунке 2.

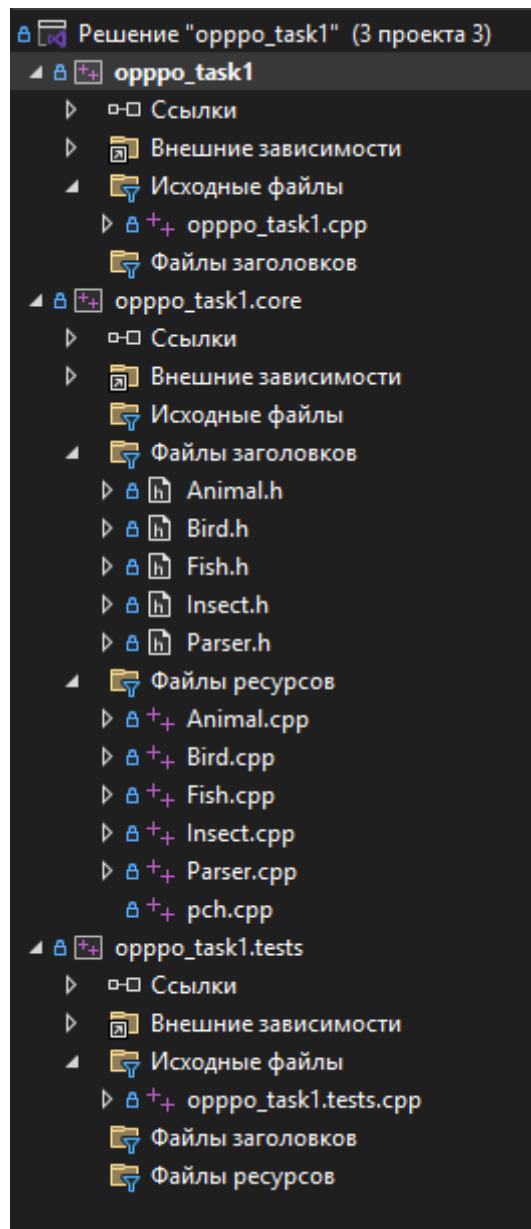


Рисунок 2 – Структура проектов

Исходя из рисунка, можно сделать вывод, что структура проекта изменилась не только благодаря добавлению подпроектов, но и благодаря разбиению ресурсных и заголовочных файлов классов. Также внутри проекта произошли изменения, повлекшие за собой уход от структуры данных вектора указателей на базовый класс к структуре вектора умных указателей в угоду избавления от утечек памяти. Остальные, более незначительные изменения, можно отследить в истории коммитов в GitHub-репозитории.

На рисунке 3 представлены заявленные workflows.

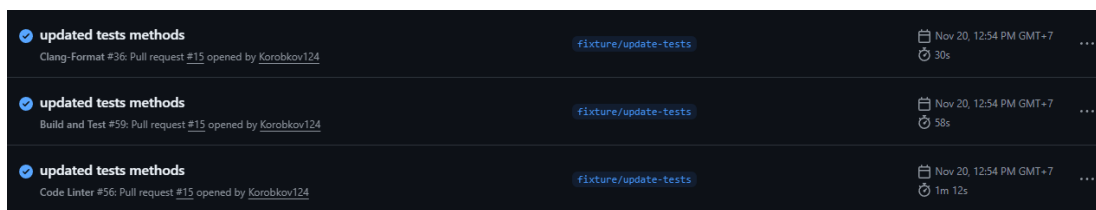


Рисунок 3 – Workflows

На рисунках 4 и 5 представлены старая и новая UML-диаграммы классов.

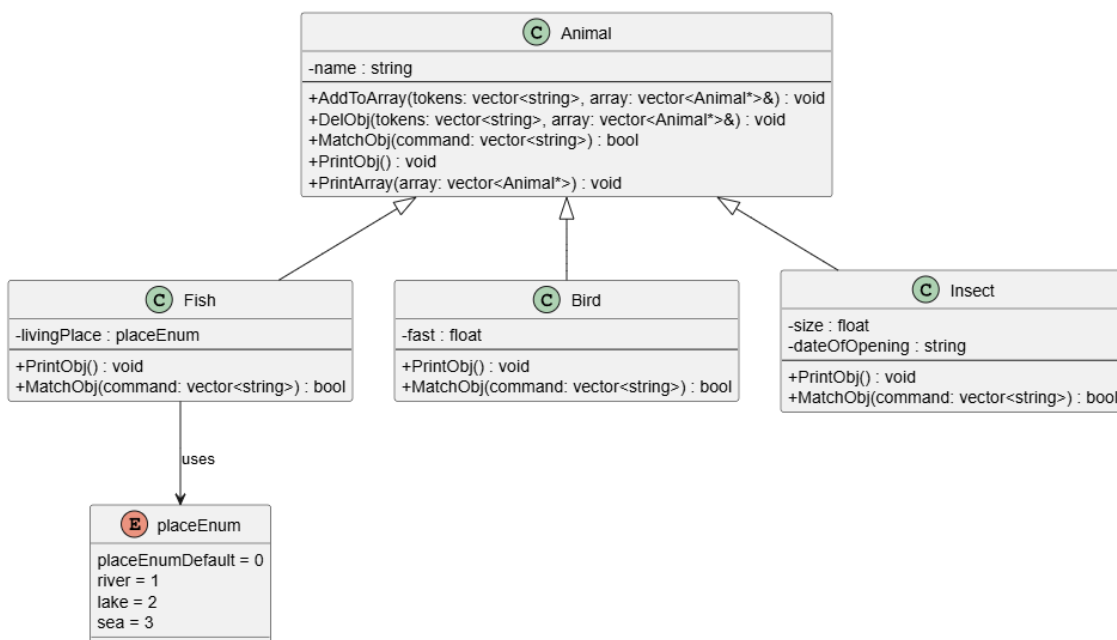


Рисунок 3 – Старая UML-диаграмма классов

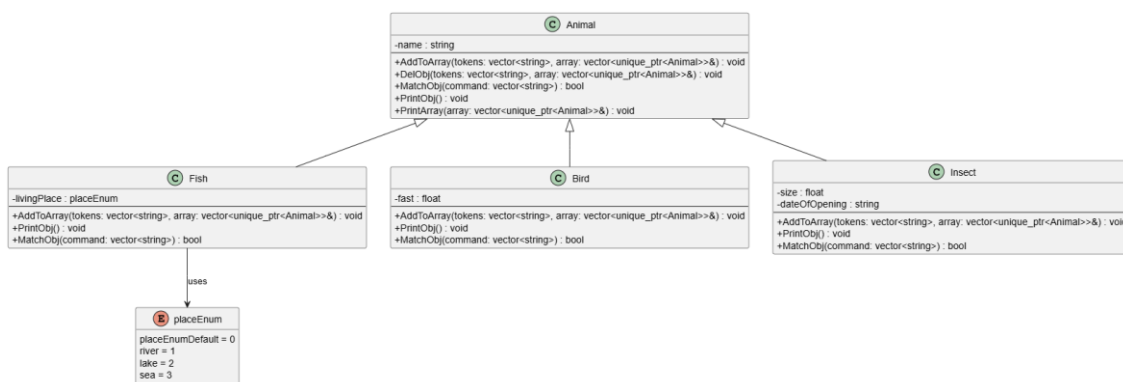


Рисунок 3 – Новая UML-диаграмма классов

ЗАКЛЮЧЕНИЕ

В результате данной практической работы были закреплены навыки рефакторинга, использования статических анализаторов кода и использования базовых принципов ООП. Были получены навыки описания СИ процесса.

ПРИЛОЖЕНИЕ А

1 GitHub. Система контроля версий: [сайт]. – URL:
<https://github.com/Korobkov124/OPPPPO>